SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:100**

Annexure A
Industry Problem Solving

Code of Conduct:

I _P Hemanth(192472151)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **P Hemanth**

RUBRICS FOR CALCULATION

SCENARIO BASED ASSESSMENT							
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	Level	Marks
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A

Question 1: Real Estate Price Prediction Using Linear Regression

A real estate company wants to predict the **selling price of houses** based on house-related features.

House	Area sq.ft	Bedrooms	Age	Distance from City km	Price Lakhs
H1	900	2	12	10	45
H2	1100	2	10	8	55
H3	1300	3	8	6	68
H4	1500	3	5	5	82
H5	1800	4	3	3	115
H6	2000	4	2	2	135
H7	1000	2	15	12	40
H8	1600	3	6	4	90

Tasks

a) Create the dataset using Python.
b) Identify independent and dependent variables.
c) Split the dataset into training and testing data.
d) Implement **Linear Regression**.
e) Predict house price for test data.
f) Predict the price of a new house with:

Area	Bedrooms	Age	Distance
1700	3	4	4

g) Evaluate the model using **MAE, MSE, RMSE, and R^2 score**.
h) Interpret the result and explain whether the model is useful for real estate price prediction.

Program:-

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Read dataset

df = pd.read_csv(r"C:\Users\Hemanth\Desktop\house_data.csv")
```

```
print("Dataset:")
print(df)
# Independent variables
X = df[["Area", "Bedrooms", "Age", "Distance"]]
# Dependent variable
y = df["Price"]
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# Create Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
print("\nActual Prices:")
print(y_test.values)
print("\nPredicted Prices:")
print(y_pred)
# Predict new house
new_house = [[1700, 3, 4, 4]]
new_price = model.predict(new_house)
print("\nNew House Price:")
print(new_price[0], "Lakhs")
# Evaluation
```

```
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("\nModel Evaluation:")

print("MAE =", mae)

print("MSE =", mse)

print("RMSE =", rmse)

print("R2 Score =", r2)
```

Output

Question 2: Customer Churn Prediction Using Logistic Regression

A telecom company wants to predict whether a customer will **churn** or **not churn**.

Customer	Monthly Bill	Complaints	Data Usage GB	Tenure Months	Churn
C1	500	0	25	36	0
C2	900	3	10	8	1
C3	750	2	15	12	1
C4	400	0	30	48	0
C5	1000	4	8	6	1
C6	550	1	22	30	0
C7	850	3	12	10	1
C8	450	0	28	40	0

Here,
Churn = 1 means customer leaves the company.
Churn = 0 means customer stays.

Tasks

a) Create the dataset using Python.
b) Identify the input features and target variable.
c) Perform train-test split.
d) Implement **Logistic Regression**.
e) Predict churn status for test customers.
f) Predict churn for a new customer:

Monthly Bill	Complaints	Data Usage GB	Tenure Months
800	2	14	10

g) Evaluate the model using **accuracy, confusion matrix, precision, recall, and F1-score**.
h) Explain why Logistic Regression is suitable for churn prediction.

Program:-

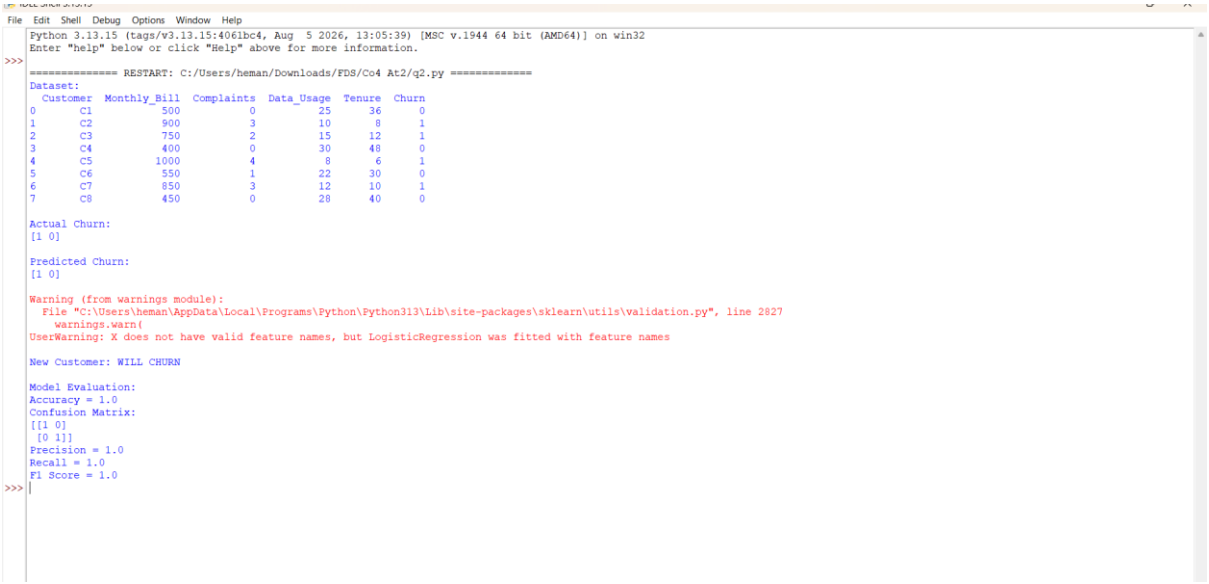
```
import pandas as pd

import numpy as np



from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix
```

```
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

# a) Read dataset
df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q2 data.csv")

print("Dataset:")
print(df)

# b) Input features and target
X = df[["Monthly_Bill", "Complaints", "Data_Usage", "Tenure"]]
y = df["Churn"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d) Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)

# e) Predict test customers
y_pred = model.predict(X_test)
```

```
print("\nActual Churn:")
print(y_test.values)

print("\nPredicted Churn:")
print(y_pred)

# f) Predict new customer
new_customer = [[800, 2, 14, 10]]

prediction = model.predict(new_customer)

if prediction[0] == 1:
    print("\nNew Customer: WILL CHURN")
else:
    print("\nNew Customer: WILL NOT CHURN")

# g) Evaluation
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)

print("\nModel Evaluation:")
```

```
print("Accuracy =", accuracy)

print("Confusion Matrix:")

print(cm)

print("Precision =", precision)

print("Recall =", recall)

print("F1 Score =", f1)
```

Output:-



Question 3: Marketing Sales Prediction Using Lasso Regression

A retail company wants to predict **monthly sales** based on advertisement spending across different platforms. The company wants to know which marketing channels are most important.

Month	TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns	Sales Lakhs
M1	50	20	10	5	15
M2	60	25	12	6	18
M3	80	40	15	8	25
M4	100	60	20	10	32
M5	120	70	22	12	38
M6	40	15	8	4	12
M7	90	50	18	9	28
M8	110	65	21	11	35

Tasks

a) Create the dataset using Python.
b) Identify features and target variable.
c) Apply train-test split.
d) Implement **Lasso Regression**.
e) Use an alpha value such as **0.1 or 1.0**.
f) Predict sales for test data.
g) Predict sales for a new campaign:

TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns
95	55	18	9

h) Evaluate using **MAE, MSE, RMSE, and R^2 score**.
i) Display the coefficients and identify which features are reduced or selected by Lasso.
j) Explain how Lasso helps in feature selection.

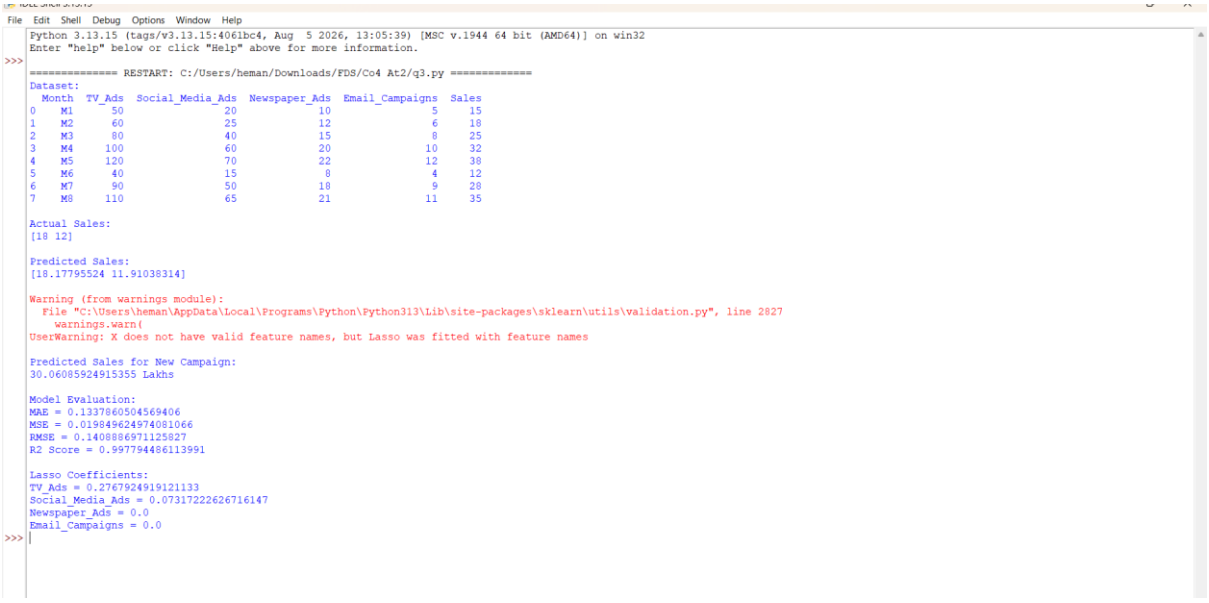
Program:-

```
import pandas as pd

import numpy as np



from sklearn.model_selection import train_test_split

from sklearn.linear_model import Lasso

from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error

from sklearn.metrics import r2_score



# a) Read dataset

df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q3.csv")



print("Dataset:")

print(df)



# b) Features and target

X = df[["TV_Ads", "Social_Media_Ads",
```

```
        "Newspaper_Ads", "Email_Campaigns"]]

y = df["Sales"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d & e) Lasso Regression
model = Lasso(alpha=0.1)

model.fit(X_train, y_train)

# f) Predict test data
y_pred = model.predict(X_test)

print("\nActual Sales:")
print(y_test.values)

print("\nPredicted Sales:")
print(y_pred)

# g) Predict new campaign
new_campaign = [[95, 55, 18, 9]]
```

```
new_sales = model.predict(new_campaign)


print("\nPredicted Sales for New Campaign:")
print(new_sales[0], "Lakhs")


# h) Evaluation
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)


print("\nModel Evaluation:")
print("MAE =", mae)
print("MSE =", mse)
print("RMSE =", rmse)
print("R2 Score =", r2)
# i) Display coefficients
print("\nLasso Coefficients:")
for feature, coefficient in zip(X.columns, model.coef_):
    print(feature, "=", coefficient)
```

Output



Question 4: Electricity Consumption Prediction Using Ridge Regression

A smart energy company wants to predict household **electricity consumption**. Some features may be highly correlated, such as number of appliances and daily usage hours. Therefore, Ridge Regression is used.

House	Appliances	Usage Hours	House Size sq.ft	Occupants	Consumption Units
H1	5	4	800	3	120
H2	7	6	1000	4	190
H3	8	7	1200	4	240
H4	10	8	1500	5	320
H5	4	3	700	2	90
H6	6	5	900	3	160
H7	9	7	1300	5	280
H8	11	9	1600	6	350

Tasks

a) Create the dataset using Python.
b) Identify the independent and dependent variables.
c) Split the data into training and testing sets.
d) Implement **Ridge Regression**.
e) Use an alpha value such as **1.0**.
f) Predict electricity consumption for test data.
g) Predict consumption for a new house:

Appliances	Usage Hours	House Size	Occupants
8	6	1100	4

h) Evaluate using **MAE, MSE, RMSE, and R^2 score**.
i) Display the Ridge coefficients.
j) Explain how Ridge Regression reduces overfitting.

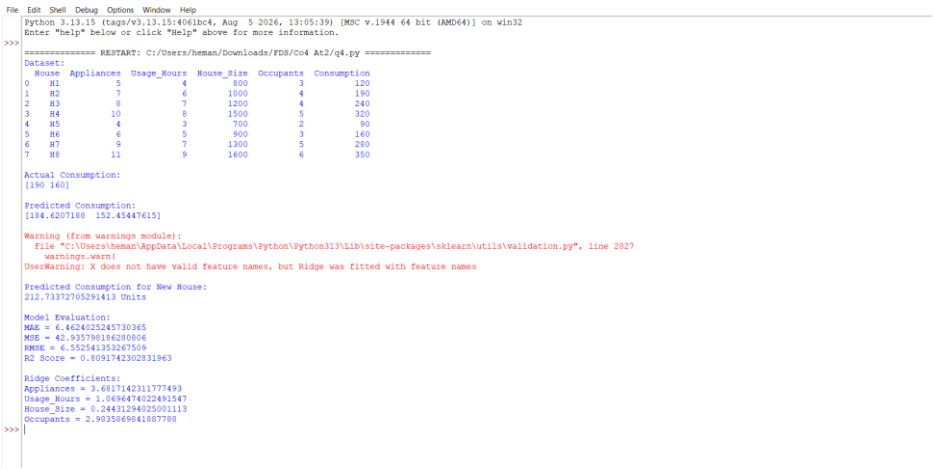
Program:-

```
import pandas as pd

import numpy as np


from sklearn.model_selection import train_test_split

from sklearn.linear_model import Ridge

from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error

from sklearn.metrics import r2_score


# a) Read dataset

df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q4 data.csv")


print("Dataset:")

print(df)


# b) Independent and dependent variables

X = df[["Appliances", "Usage_Hours", "House_Size", "Occupants"]]


y = df["Consumption"]


# c) Split dataset

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.25, random_state=42
)

# d & e) Ridge Regression
model = Ridge(alpha=1.0)

model.fit(X_train, y_train)

# f) Predict test data
y_pred = model.predict(X_test)

print("\nActual Consumption:")
print(y_test.values)

print("\nPredicted Consumption:")
print(y_pred)

# g) Predict new house
new_house = [[8, 6, 1100, 4]]

new_consumption = model.predict(new_house)

print("\nPredicted Consumption for New House:")
print(new_consumption[0], "Units")
```

```
# h) Evaluation

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)


print("\nModel Evaluation:")

print("MAE =", mae)

print("MSE =", mse)

print("RMSE =", rmse)

print("R2 Score =", r2)


# i) Display Ridge coefficients

print("\nRidge Coefficients:")


for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, "=", coefficient)
```

Output:-



Question 5: Delivery Time Prediction Using Linear Regression

A logistics company wants to predict **delivery time** based on distance, traffic level, number of packages, and weather score.

Delivery	Distance km	Traffic Level	Packages	Weather Score	Time Hours
D1	10	2	20	8	1.2
D2	20	4	30	7	2.4
D3	30	6	40	6	3.5
D4	40	8	50	5	4.8
D5	50	9	60	4	6.0
D6	15	3	25	8	1.8
D7	35	7	45	5	4.2
D8	25	5	35	6	3.0

Tasks

a) Create the dataset.
b) Identify input and output variables.
c) Split the data into train and test sets.
d) Implement **Linear Regression**.
e) Predict delivery time for test records.
f) Predict delivery time for:

Distance	Traffic Level	Packages	Weather Score
28	6	38	6

g) Evaluate using **MAE, MSE, RMSE, and R^2 score**.
h) Explain how this model can help a logistics company plan delivery schedules.

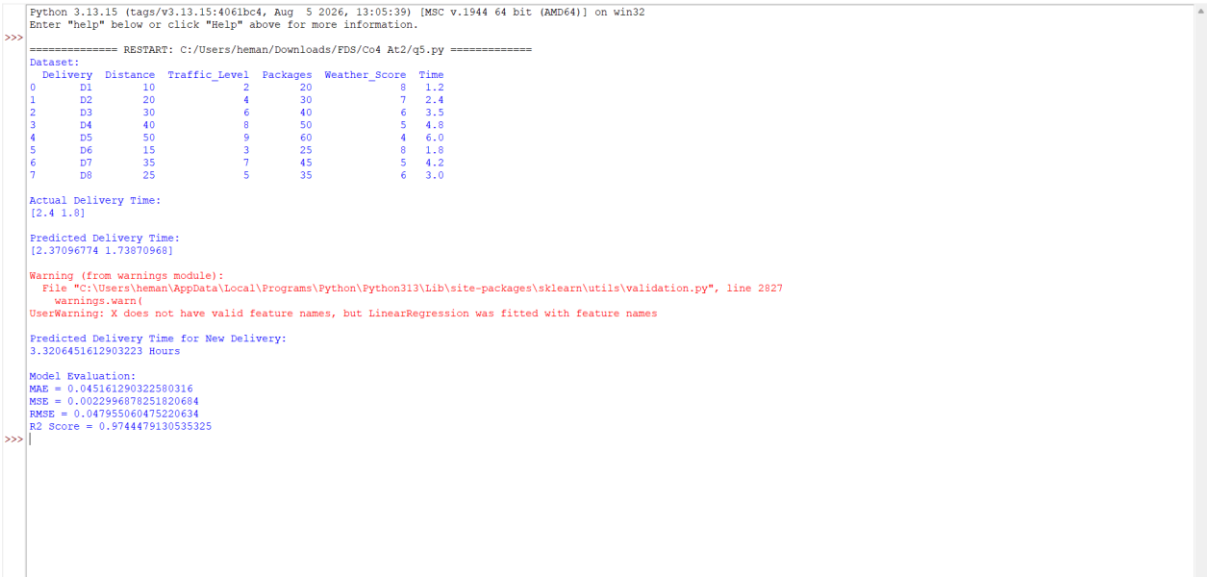
Program:-

```
import pandas as pd

import numpy as np



from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error

from sklearn.metrics import r2_score
```

```
# a) Read dataset
df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q5 data.csv")

print("Dataset:")
print(df)

# b) Input and output variables
X = df[["Distance", "Traffic_Level", "Packages", "Weather_Score"]]

y = df["Time"]

# c) Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# d) Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# e) Predict test records
y_pred = model.predict(X_test)

print("\nActual Delivery Time:")
```

```
print(y_test.values)


print("\nPredicted Delivery Time:")
print(y_pred)


# f) Predict new delivery
new_delivery = [[28, 6, 38, 6]]


new_time = model.predict(new_delivery)


print("\nPredicted Delivery Time for New Delivery:")
print(new_time[0], "Hours")


# g) Evaluation
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)


print("\nModel Evaluation:")
print("MAE =", mae)
print("MSE =", mse)
print("RMSE =", rmse)
print("R2 Score =", r2)
```

Output



Question 6: Loan Approval Prediction Using Logistic Regression

A bank wants to predict whether a loan application should be **approved** or **rejected**.

Applicant	Income	Credit Score	Existing Loan	Employment Years	Approved
A1	25000	580	200000	1	0
A2	30000	600	180000	2	0
A3	45000	680	120000	3	1
A4	60000	750	80000	5	1
A5	70000	800	50000	6	1
A6	28000	590	220000	1	0
A7	55000	720	100000	4	1
A8	35000	620	160000	2	0

Here,
Approved = 1 means loan approved.
Approved = 0 means loan rejected.

Tasks

a) Create the dataset in Python.
b) Identify independent and dependent variables.
c) Split the dataset.
d) Implement **Logistic Regression**.
e) Predict approval status for test data.
f) Predict loan approval for a new applicant:

Income	Credit Score	Existing Loan	Employment Years
50000	700	100000	4

g) Evaluate using **accuracy, confusion matrix, precision, recall, and F1-score**.
h) Explain why Logistic Regression is suitable for loan approval prediction.

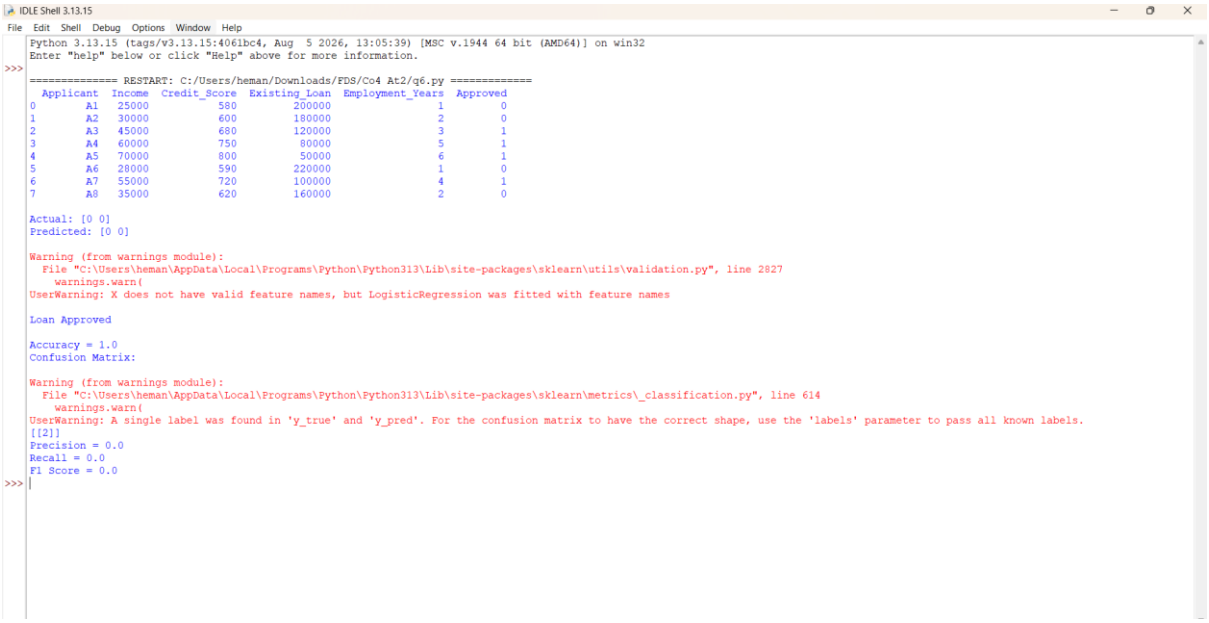
Program:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import precision_score, recall_score, f1_score

# Read dataset
df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q6 data.csv")

print(df)

# Independent variables
X = df[["Income", "Credit_Score", "Existing_Loan",
        "Employment_Years"]]

# Dependent variable
y = df["Approved"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Logistic Regression
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Test prediction
y_pred = model.predict(X_test)

print("\nActual:", y_test.values)
print("Predicted:", y_pred)

# New applicant
new_applicant = [[50000, 700, 100000, 4]]

prediction = model.predict(new_applicant)

if prediction[0] == 1:
    print("\nLoan Approved")
else:
    print("\nLoan Rejected")

# Evaluation
print("\nAccuracy =", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Precision =", precision_score(y_test, y_pred, zero_division=0))
print("Recall =", recall_score(y_test, y_pred, zero_division=0))
```

print("F1 Score =", f1_score(y_test, y_pred, zero_division=0))

Output:-



Question 7: Healthcare Treatment Cost Prediction Using Lasso Regression

A hospital wants to predict **treatment cost** based on patient health parameters. Since many medical features are available, the hospital wants to use Lasso Regression to identify important predictors.

Patient	Age	BP	Sugar Level	BMI	Previous Visits	Treatment Cost
P1	25	115	90	22	1	5000
P2	35	125	110	25	2	8000
P3	45	140	150	29	3	15000
P4	55	155	180	32	5	25000
P5	60	165	200	35	6	32000
P6	30	120	100	24	1	7000
P7	50	150	170	31	4	22000
P8	40	135	130	27	2	12000

Tasks

a) Create the dataset using Python.
b) Identify features and target variable.
c) Split into training and testing sets.
d) Implement **Lasso Regression**.
e) Try alpha values such as **0.1, 1.0, and 10.0**.
f) Predict treatment cost for test patients.
g) Predict treatment cost for a new patient:

Age	BP	Sugar Level	BMI	Previous Visits
48	145	160	30	4

h) Evaluate using **MAE, MSE, RMSE, and R^2 score**.
i) Print the model coefficients.
j) Explain which medical features are important based on Lasso coefficients.

Program:-

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Lasso

from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error, r2_score



# Read dataset

df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q7 data.csv")



print(df)



# Features

X = df[["Age", "BP", "Sugar_Level", "BMI", "Previous_Visits"]]



# Target

y = df["Treatment_Cost"]



# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.25, random_state=42
```

```
)

# Try different alpha values
for alpha in [0.1, 1.0, 10.0]:

    model = Lasso(alpha=alpha, max_iter=10000)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    print("\nAlpha =", alpha)

    print("MAE =", mean_absolute_error(y_test, y_pred))

    mse = mean_squared_error(y_test, y_pred)
    print("MSE =", mse)

    print("RMSE =", np.sqrt(mse))

    print("R2 =", r2_score(y_test, y_pred))

    print("Coefficients:")
    for feature, coefficient in zip(X.columns, model.coef_):
        print(feature, "=", coefficient)
```

New patient using alpha 0.1

model = Lasso(alpha=0.1, max_iter=10000)

model.fit(X_train, y_train)

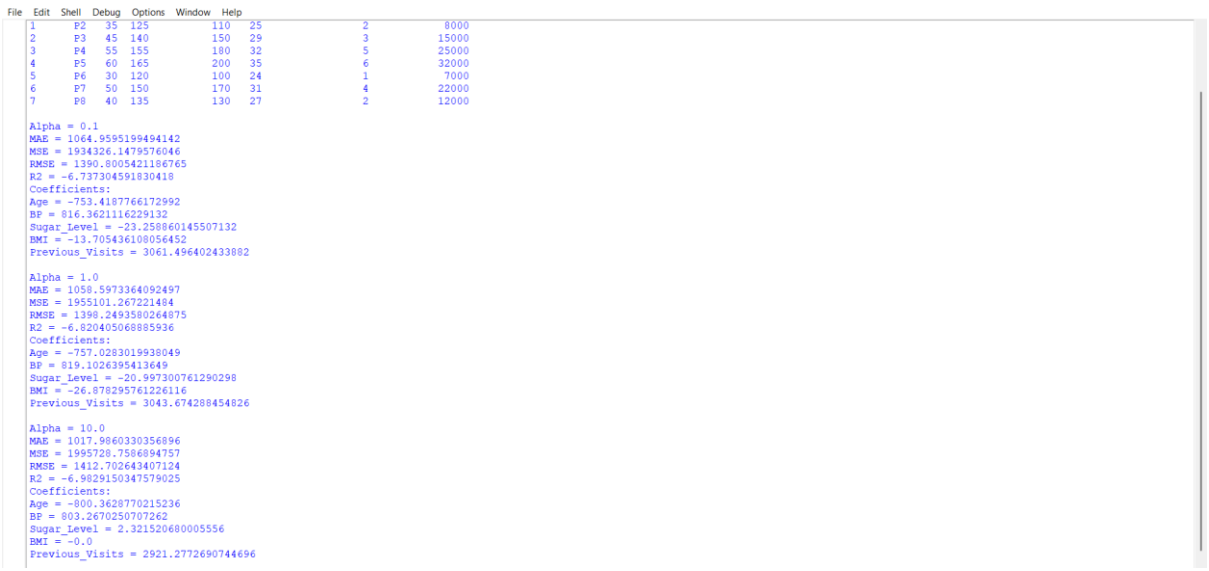
new_patient = [[48, 145, 160, 30, 4]]

prediction = model.predict(new_patient)

print("\nPredicted Treatment Cost:")

print(prediction[0])

Output:-



Question 8: Manufacturing Defect Strength Prediction Using Ridge Regression

A manufacturing company wants to predict the **strength score** of a product based on machine parameters. Since temperature, pressure, and machine speed may be correlated, Ridge Regression is used.

Product	Temperature	Pressure	Machine Speed	Material Quality	Strength Score
P1	70	30	100	8	75
P2	75	35	110	7	78
P3	80	38	120	6	80

Product	Temperature	Pressure	Machine Speed	Material Quality	Strength Score
P4	85	42	125	6	82
P5	90	45	130	5	79
P6	95	50	140	4	76
P7	78	36	115	7	81
P8	88	44	128	5	80

Tasks

a) Create the dataset.
b) Identify the independent variables and dependent variable.
c) Split into training and testing data.
d) Implement **Ridge Regression**.
e) Use alpha values such as **0.1, 1.0, and 10.0**.
f) Predict strength score for test products.
g) Predict strength score for a new product:

Temperature	Pressure	Machine Speed	Material Quality
82	40	122	6

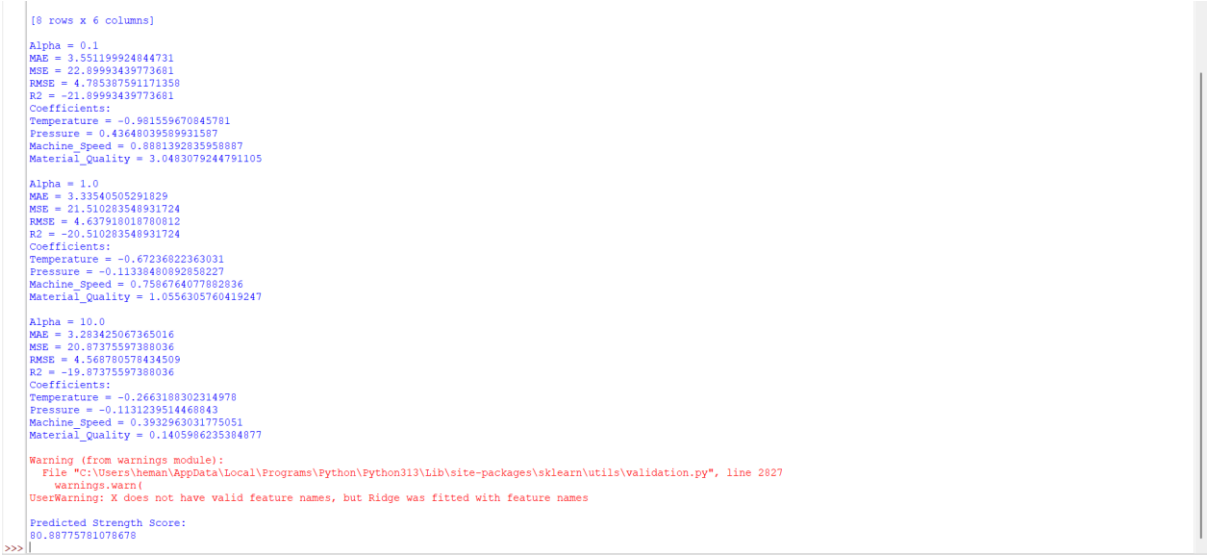
h) Evaluate using **MAE, MSE, RMSE, and R^2 score**.
i) Display Ridge coefficients.
j) Explain how Ridge handles multicollinearity in manufacturing data.

```
import pandas as pd

import numpy as np



from sklearn.model_selection import train_test_split

from sklearn.linear_model import Ridge

from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error, r2_score



# Read dataset

df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q8 data.csv")



print(df)
```

```
# Independent variables
X = df[["Temperature", "Pressure",
        "Machine_Speed", "Material_Quality"]]

# Dependent variable
y = df["Strength_Score"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Try alpha values
for alpha in [0.1, 1.0, 10.0]:

    model = Ridge(alpha=alpha)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    print("\nAlpha =", alpha)

    print("MAE =", mean_absolute_error(y_test, y_pred))

    mse = mean_squared_error(y_test, y_pred)
```

```
    print("MSE =", mse)

    print("RMSE =", np.sqrt(mse))

    print("R2 =", r2_score(y_test, y_pred))

    print("Coefficients:")

    for feature, coefficient in zip(X.columns, model.coef_):
        print(feature, "=", coefficient)

# New product using alpha = 1.0
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)

new_product = [[82, 40, 122, 6]]

prediction = model.predict(new_product)

print("\nPredicted Strength Score:")
print(prediction[0])
```

Output:-



```
[8 rows x 6 columns]

Alpha = 0.1
MAE = 3.551199924844731
MSE = 22.899934399773681
RMSE = 4.785387591171358
R2 = -21.899934399773681
Coefficients:
Temperature = -0.981559670845781
Pressure = 0.436480395899315 87
Machine_Speed = 0.8881392835958887
Material_Quality = 3.0483079244791105

Alpha = 1.0
MAE = 3.33540505291829
MSE = 21.510283548931724
RMSE = 4.637918018780812
R2 = -20.510283548931724
Coefficients:
Temperature = -0.672368223630 31
Pressure = -0.11338480892858227
Machine_Speed = 0.7586764077882836
Material_Quality = 1.0556305760419247

Alpha = 10.0
MAE = 3.283425067365016
MSE = 20.873755973880 36
RMSE = 4.568780578434509
R2 = -19.873755973880 36
Coefficients:
Temperature = -0.2663188302314978
Pressure = -0.1131239514468843
Machine_Speed = 0.3932963031775051
Material_Quality = 0.1405986235384877

Warning (from warnings module):
  File "C:\Users\heman\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\validation.py", line 2827
    warnings.warn(
UserWarning: X does not have valid feature names, but Ridge was fitted with feature names

Predicted Strength Score:
80.88775781078678
>>>
```

Question 9: Student Salary Package Prediction Using Linear Regression

A university placement cell wants to predict the **expected salary package** of students.

Student	CGPA	Aptitude Score	Coding Score	Communication Score	Package LPA
S1	6.5	60	55	65	3.2
S2	7.0	65	60	70	4.0
S3	7.5	70	68	72	5.0
S4	8.0	78	80	75	6.5
S5	8.5	85	88	80	8.0
S6	9.0	90	92	85	10.0
S7	6.8	62	58	68	3.8
S8	8.2	80	82	78	7.0

Tasks

a) Create the dataset using Python.
b) Identify input and output variables.
c) Split the dataset.
d) Implement **Linear Regression**.
e) Predict package for test students.
f) Predict package for a new student:

CGPA	Aptitude Score	Coding Score	Communication Score
8.1	82	85	78

g) Evaluate using **MAE, MSE, RMSE, and R^2 score**.
h) Interpret the result from the placement cell perspective.

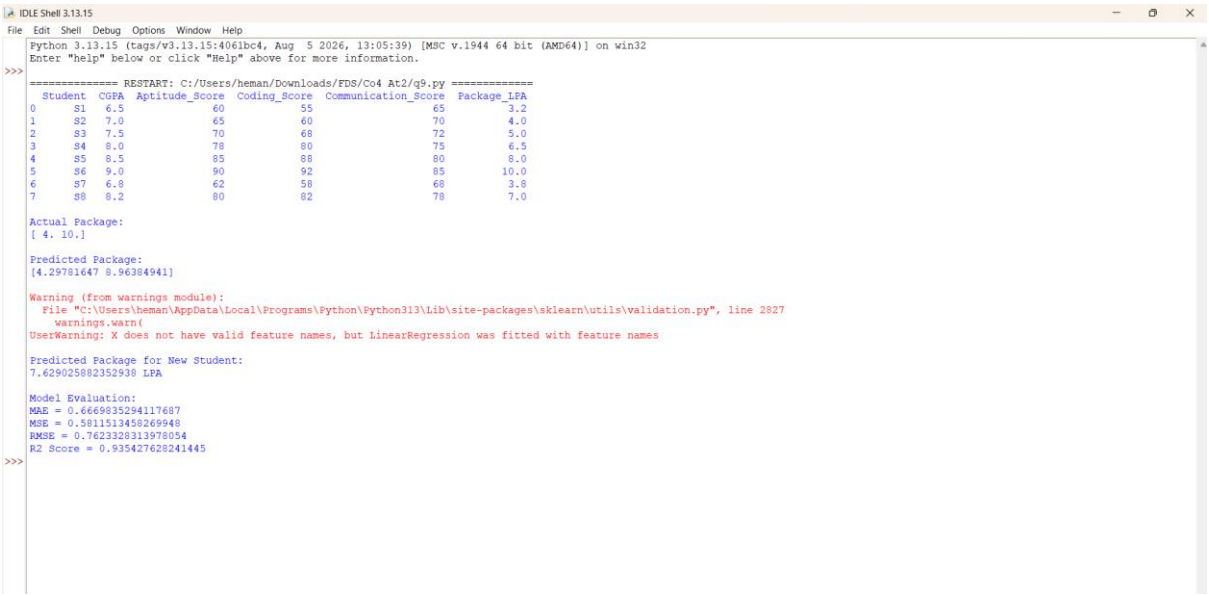
Program:-

```
import pandas as pd

import numpy as np


from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error, r2_score


# Read dataset

df = pd.read_csv(r"C:\Users\heman\Downloads\FDS\Co4 At2\q9 data.csv")


print(df)


# Input variables

X = df[["CGPA", "Aptitude_Score",

        "Coding_Score", "Communication_Score"]]


# Output variable

y = df["Package_LPA"]


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.25, random_state=42

)
```

```
# Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)

# Test prediction
y_pred = model.predict(X_test)

print("\nActual Package:")
print(y_test.values)

print("\nPredicted Package:")
print(y_pred)

# New student
new_student = [[8.1, 82, 85, 78]]

prediction = model.predict(new_student)

print("\nPredicted Package for New Student:")
print(prediction[0], "LPA")

# Evaluation
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)


print("\nModel Evaluation:")

print("MAE =", mae)

print("MSE =", mse)

print("RMSE =", rmse)

print("R2 Score =", r2)
```

Output:-



Question 10: Employee Attrition Prediction Using Logistic Regression

An HR department wants to predict whether an employee will **leave** the company or **stay**.

Employee	Experience	Satisfaction Score	Overtime Hours	Salary Increment %	Leave
E1	2	8	2	12	0
E2	3	7	4	10	0
E3	1	4	10	5	1
E4	5	6	6	8	0
E5	6	3	12	4	1
E6	8	2	14	3	1
E7	4	7	3	11	0

Employee	Experience	Satisfaction Score	Overtime Hours	Salary Increment %	Leave
E8	7	4	11	5	1

Here,
Leave = 1 means employee may leave.
Leave = 0 means employee may stay.

Tasks

a) Create the dataset using Python.
b) Identify features and target variable.
c) Split into training and testing data.
d) Implement **Logistic Regression**.
e) Predict attrition for test employees.
f) Predict attrition for a new employee:

Experience	Satisfaction Score	Overtime Hours	Salary Increment %
5	4	9	6

g) Evaluate using **accuracy, confusion matrix, precision, recall, and F1-score**.
h) Explain how the HR team can use this prediction to reduce employee attrition.

Program:-

```
import pandas as pd



from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix

from sklearn.metrics import precision_score

from sklearn.metrics import recall_score

from sklearn.metrics import f1_score



# Read dataset

df = pd.read_csv(

    r"C:\Users\heman\Downloads\FDS\Co4 At2\q10 data.csv"
```

```
)

print(df)

# Features
X = df[["Experience", "Satisfaction_Score",
        "Overtime_Hours", "Salary_Increment"]]

# Target
y = df["Leave"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Test prediction
y_pred = model.predict(X_test)

print("\nActual Leave:")
print(y_test.values)
```

```
print("\nPredicted Leave:")
print(y_pred)

# New employee
new_employee = [[5, 4, 9, 6]]

prediction = model.predict(new_employee)

if prediction[0] == 1:
    print("\nEmployee may LEAVE")
else:
    print("\nEmployee may STAY")

# Evaluation
print("\nModel Evaluation:")

print("Accuracy =",
      accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("Precision =",
      precision_score(y_test, y_pred, zero_division=0))
```

```
print("Recall =",
      recall_score(y_test, y_pred, zero_division=0))

print("F1 Score =",
      f1_score(y_test, y_pred, zero_division=0))
```

Output:-